

```

        else
        {
            return 0; /*return to the
func1()*/
        }
    }
}
filegen()
1197 -> {
    FILE *fo,*fg;
    int f=0;
    fo=fopen("/root/tempc.txt","a");
    while(f<4000)
    {
        fprintf(fo,"%d\n",f);
        f++;
    }
    fclose(fo);
    system("md5sum /root/tempc.txt >> /root/
hashes.txt");/*Unique MD5 hash generation*/
}
func3()
9601 -> {
    int l=0,b=0;
    while(l<=50000)
    {
        b=l;
        l++;
    }
    if(l==50001)
    {
        counter3++;
    }
}

```

Top 10 Lines:

Line Count

```

72 9601
59 1197
32 399
43 399

```

Execution Summary:

```

5 Executable lines in this file
5 Lines executed
100.00 Percent of the file executed

11596 Total number of line executions
2319.20 Average executions per line

```

The timing presented is by user-space; so the system calls generally spend far less time in user-space as compared to kernel-space. That's why we see the 0.00 in the Time collection of Gprof. From the SAR (system activity report) snapshot attached, it can be seen that system space is using around 94 per cent of the CPU processing, as compared to 5-6 per cent of the user space.

That's it, guys; happy coding! I will be back with some more cool stuff. 

References

Gprof: a Call Graph Execution Profiler* by Susan L. Graham, Peter B. Kessler, Marshall K. McKusick
<http://docs.freebsd.org/44doc/psd/18.gprof/paper.pdf>

By: Subodh V Pachghare

The author works with India's largest commercial automotive organisation, in the IT division. He is interested in the networking, information security, back-up and virtualisation domains. He is a FOSS follower, and spends most of his time learning the internals of Linux. You can reach him at www.thesubodh.com or email him at subodhpachghare@gmail.com.

THE COMPLETE MAGAZINE ON OPEN SOURCE

LINUX BETA
ForYou

Your favourite Linux Magazine is
 now on the Web, too.

LinuxForU.com

Follow us on Twitter @LinuxForYou